TripMate AI - Technical Documentation

A multi-agent travel planner for **worldwide** trips. A natural-language request becomes a full plan (flights/transport, hotels, weather, day-by-day itinerary, budget) produced by a team of AI agents orchestrated on **Google ADK**, protected by Responsible-AI guardrails, and scored by an automated evaluator. **Free and local** by default (Ollama + free APIs + SQLite).

> For a deep per-agent + ADK + evaluation walkthrough see **AGENTS_AND_ADK.md**.

1. Architecture

```
user query
   |
   v
Input Guardrail  (safety, injection, relevance; Llama Guard + rules)
   | pass
   v
Supervisor  (extract trip constraints, enforce scope, pick agents dynamically)
   |
   v
Human Review  (optional approve/cancel)
   |
   +-> Flight   Hotel   Weather   Budget   Itinerary   (selected subset, parallel)
   |
   v
Final Response  (compose the plan)
   |
   v
Evaluation  (heuristic metrics + optional LLM judge > score)

State - sessions - memory: Google ADK SessionService (durable SQLite) + SQLite stores.
```

2. Technology - what and where

Technology	Role	Where
Google ADK (`google-adk`)	Orchestration/runtime: sessions, state, events, memory	`travel_agent/orchestrator.py`, `memory.py`
Ollama (`llama3.2`) / **Groq** (free tier)	The LLM for all agents	`travel_agent/llm.py`
MCP (`mcp`)	Tools exposed as MCP servers	`mcp_servers/*.py`, `travel_agent/mcp_client.py`
Open-Meteo (free, no key)	Weather + geocoding (worldwide)	`travel_agent/tools/weather.py`
AviationStack / Tavily / Google Places	Flights / web search / places (optional keys)	`travel_agent/tools/{flights,search,places`
airportsdata + pycountry	Worldwide airport & country resolution	`travel_agent/india.py`
Streamlit	Web UI	`streamlit_app.py`
SQLite (stdlib)	Preferences, long-term memory, sessions, scores, user accounts	`memory.py`, `auth.py`, `evaluation` logging
markdown + xhtml2pdf	Plan/doc export to PDF	`scripts/md2pdf.py`, `travel_agent/export.py`
Llama Guard (free)	Safety moderation	`travel_agent/responsible_ai.py`

Technology	Role	Where
pytest	Automated tests	`tests/`

3. Components

- **Orchestrator** (`orchestrator.py`) - `TravelPlanner`: drives the workflow over an ADK `SessionService`, merges `TravelState` with reducers, streams `(node, update)`, handles HITL pause/resume, runs evaluation, and saves memory.
- **State** (`state.py`) - `TravelState` TypedDict + Pydantic models (`TripDetails`, `SupervisorDecision`, `GuardrailVerdict`, `JudgeScore`).
- **Agents** (`agents/`) - guardrail, supervisor, human_review, the 5 specialists, final. Specialists call tools via MCP and summarise with the LLM; each output is Responsible-AI guarded.
- **Responsible AI** (`responsible_ai.py`) - PII redaction, injection detection, Llama-Guard/rules safety, grounding disclaimers, `rai_log` transparency; `with_output_guardrail` wraps every agent.
- **Evaluation** (`evaluation.py`) - heuristic metrics (completeness, safety, grounding, personalization, efficiency) + optional LLM-as-judge > `eval_report`.
- **Memory** (`memory.py`) - ADK session service (durable SQLite), `PreferenceStore`, `MemoryStore`, `ScoreLog`.
- **Scope** (`india.py`) - worldwide airport resolution, country lookup, and the `TRAVEL_SCOPE` policy.
- **Auth** (`auth.py`, `login_ui.py`) - SQLite accounts with PBKDF2-hashed passwords.

4. Run & configure

```
.venv\Scripts\python -m streamlit run streamlit_app.py   # http://localhost:8502
```

Key `.env` settings: `LLM_PROVIDER`, `GROQ_API_KEY`, `TRAVEL_SCOPE` (global/india), `ENABLE_RAI`, `ENABLE_LLM_JUDGE`, `USE_MCP`, `ADK_SQLITE_SESSIONS`, optional tool keys. Offline scoring: `python evaluate_suite.py`. Tests: `pytest`.

5. Notes

- Orchestration is **Google ADK**; LangGraph was removed from the app code.
- Tools degrade gracefully without optional keys; the plan is still produced.
- All state persists locally in SQLite (git-ignored). No paid services are required.